Architecture Discoverability and Verification

Agata Przybyszewska, PhD

Senior Enterprise Architect · DSV

Architect, DevSecOps engineer, Coder · MathMagicians

Faglig Fredag · 30 October 2026

Background

Big enterprise, stale docs

What problem are we trying to solve?

Compliance, audit, governance.

Stop gates - you want it deployed? You need approval.

Then … life goes on[1] …



1. *The author believes that the thing worse than no documentation, is stale documentation.*

Must architecture docs always go stale?

So what is Architecture Discoverability?

Three questions

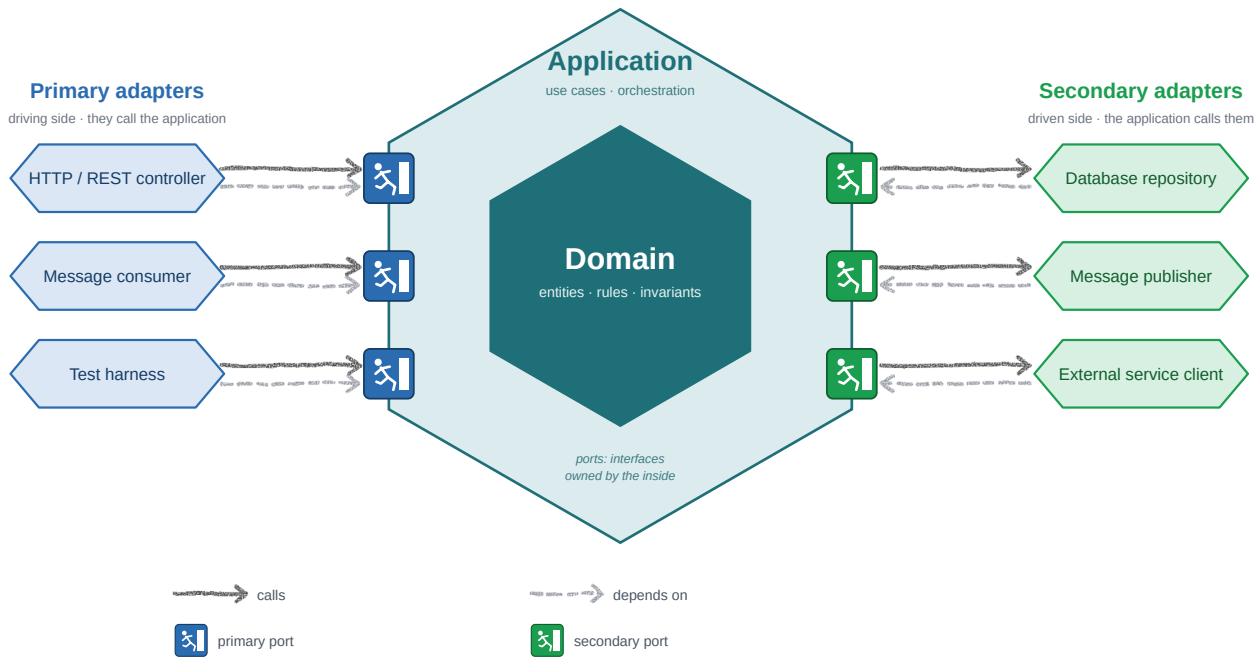
How do we specify it?

How do we keep it in sync, as life goes on?

How do we verify it?

Hexagonal Architecture

This style will be used in our examples.



Meet Wonderland, a toy platform

Built to explore the core ideas

Wonderland plays stories that send messages to each other; CLI, Kafka, and Confluent are the transports, the stories the same on each

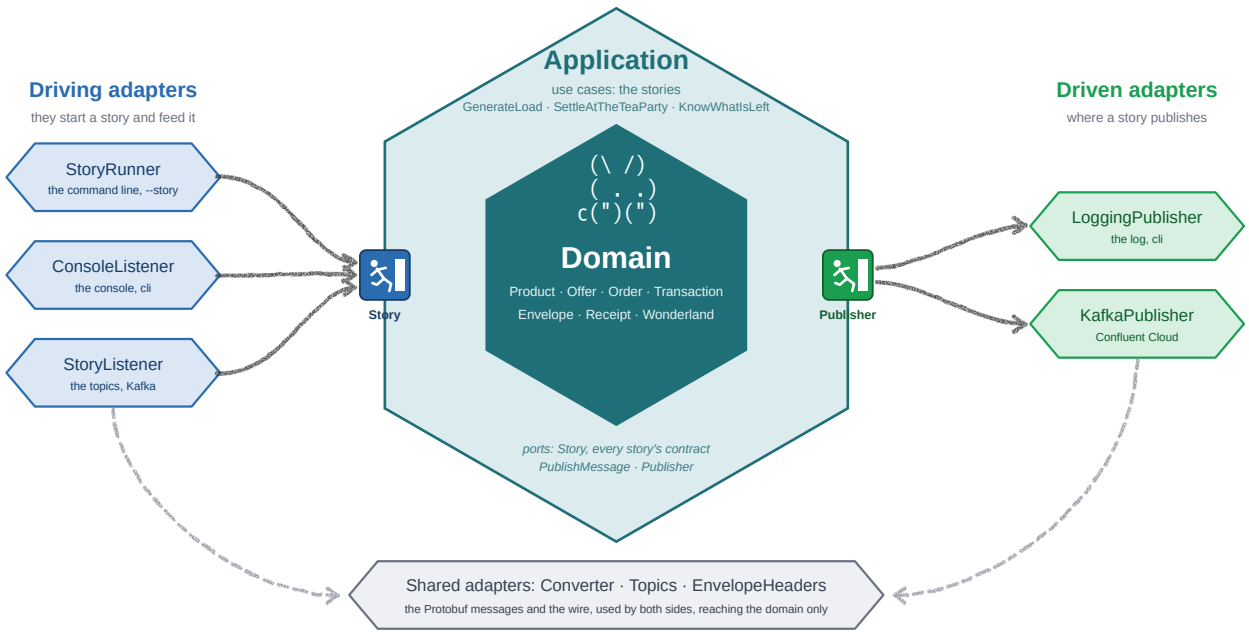
One Spring Boot substrate plays every story, `--story=<name>`, one image, one process per story

Platform Engineering is a skill

DX, make it easy - red carpet to do the right thing

Keep it simple, hide away complexity and deep tech stacks

Wonderland, as planned



Wonderland - Tech stack

Why Java? Enterprise

μ-services: Java 25, Spring Boot, etc ...

Runtime: JRE, Docker + Docker Compose to run it

Eventing Platforms: Kafka, Confluent[1] , Cloudflare, CLI

Testing: TDD: JUnit, BDD: Cucumber, Testcontainers

IaC: Terraform, Terraform Cloud

SDLC: GitHub Actions, Makefile, Gradle[2]

Architecture discovery: ArchUnit, JMolecules, PlantUML + our PlantUML plugin

1. *expensive*
2. *... and the slides you see run from a cosy Docker container with Slidev*

Java 25, Spring Boot, Docker

μ-services: Java 25, Spring Boot + Runtime: JRE, Docker + Docker Compose

		Run
Java 25 + Spring Boot EventingPlatform.java	The Spring Boot application is the substrate for the ports and the adapters; a process plays one story	`make run ARGS="--story=tea-party"`
Docker + Compose compose.yaml	Starts the flock from one image: a generator per region and the tea party	`TTL=300 make up`

Kafka, Confluent, CLI

Eventing Platforms: Kafka, Confluent, Cloudflare, CLI

		Run
CLI ConsoleListener.java	Without a cluster: a story publishes to the log and hears what is typed on stdin	`make run ARGS="--story=tea-party --tea-party.ttl="`
The kafka universe compose.kafka.yaml	A broker, a schema registry, and a topic viewer on the machine, with the topics and schemas `iac/` gives Confluent Cloud; what the `bdd` job plays on	`make kafka-universe-up` · `make run-tiny TRANSPORT=kafka`
Kafka on Confluent Cloud KafkaPublisher.java · kafka.properties	Records carry CloudEvents headers and a Protobuf payload; the key picks the partition	`make run-tiny TRANSPORT=confluent`

Development methodology

Testing: TDD: JUnit, BDD: Cucumber, Testcontainers

		Run
TDD TransactionTest.java · ArchitectureTest.java	Unit tests with JUnit 5, AssertJ, and Mockito, one class per production class; the architecture rules are unit tests too	`make test`
BDD i_can_generate_events.feature · StreamSteps.java	Cucumber features and step definitions; each feature plays against the image in Testcontainers, a red feature blocks the build	`make bdd`
Spec Driven How we work · discovery.md	Discovery → Formulation → Automation → Verification; a release plan per folder, the board follows it	`make gh-project-sync`

The DevSecOps part

IaC: Terraform, Terraform Cloud + SDLC: GitHub Actions, Makefile, Gradle

		Run
Terraform + Terraform Cloud topics.tf	Topics, schemas, and Flink tables, applied on every push to `main`	`make tf-plan`
GitHub Actions cicd.yaml	Makefile drives the GitHub workflow, commands can be run locally	`make check`
Makefile Makefile	All commands work locally, and drive the CICD workflows.	`make help`
Gradle platform.gradle	`platform.gradle` holds the conventions every build shares; a story's own build applies it too	`make build`

Architecture discovery kit

Architecture discovery: ArchUnit, jMolecules, Spring Modulith, PlantUML + our own tool, architecture-discovery

		Run
jMolecules Story.java · package-info.java	The stereotypes on the code: `@Module` on a package, `@PrimaryPort`, `@SecondaryPort`, `@PrimaryAdapter`, `@SecondaryAdapter` on the types; the annotation is the spec	
ArchUnit + Spring Modulith HexagonalArchitecture.java · ModuleDocumentation.java	The hexagonal rule and the module boundaries as unit tests, red when the core reaches an adapter; a client's build gets them for free	`make arch-verify`
architecture-discovery, our own tool hexagon.proto · ModelWriter.java	Reads the classes once into one model and writes the DSL, `<story>.textproto`; draws the three levels from the model, the forest, the streams, the hexagon	`make arch-gen`
PlantUML hexagon.style.puml	Renders the three drawings to SVG, the content from the model, the look in one style file	`make arch-gen`
The checks ModelCheck.java · HexagonSteps.java	The DSL as specified against the DSL as built; the architecture features in Gherkin against the model, on the steps the platform ships	`make arch-check` · `make arch-bdd`

First iteration

cli, confluent

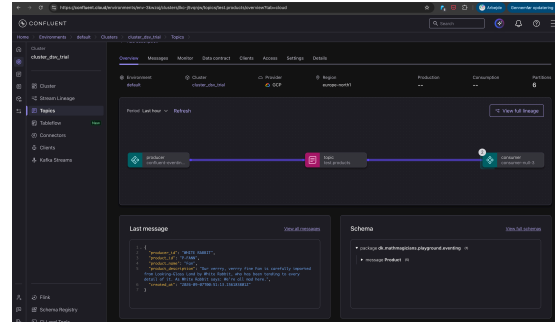
cli: the console stands in for the topics, the log is where a story publishes, stdin is what it hears

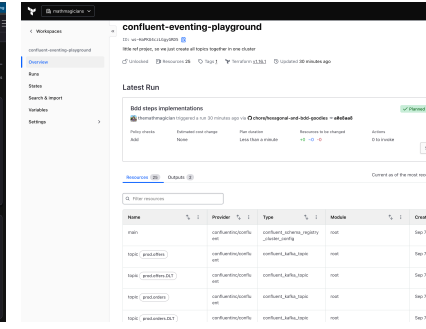
```
make run ARGS="--story=tea-party --tea-party.ttl="
Offer offer_id: "OFF-1" product_id: "P-TOPH" price: 12.5 sel
Order id: "ORD-1" customer_id: "ALICE" product_id: "P-TOPH"
```

```
12:00:03.117  INFO [main] SettleAtTheTeaParty : tea-party: S
12:00:03.121  INFO [main] LoggingPublisher : Published 7c2e-
```

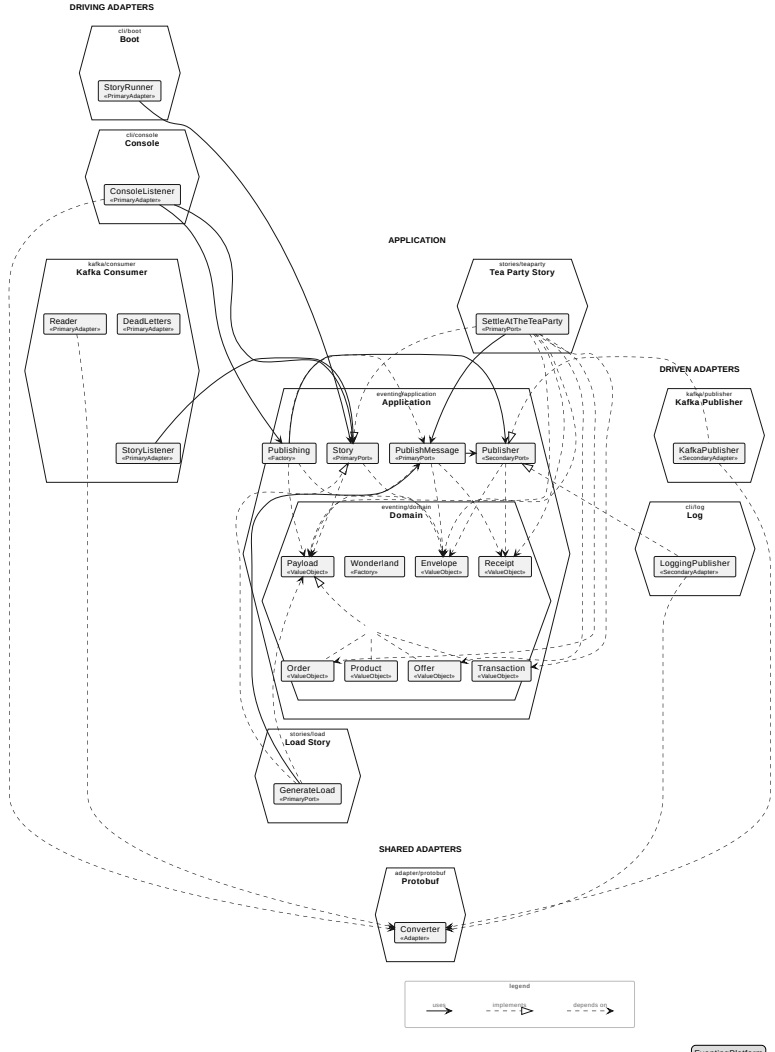
confluent: Confluent Cloud, the topics, schemas, and Flink tables from `iac/`, applied by Terraform Cloud; the keys from `.env*`

```
make tf-plan
make run-tiny TRANSPORT=confluent
```





● REC · cli, confluent



The wiring: a transport is a profile

A process plays `<transport>,<env>` ; the adapters of a transport carry it, the properties of a transport connect it

```
@PrimaryAdapter @Component @Profile("cli")        public final class ConsoleListener ...    // adapter/cli/console
@SecondaryAdapter @Component @Profile("cli")      final class LoggingPublisher ...          // adapter/cli/log
@PrimaryAdapter @Component @Profile("confluent")  public final class StoryListener ...      // adapter/kafka/consumer
@SecondaryAdapter @Component @Profile("confluent") final class KafkaPublisher ...          // adapter/kafka/publisher
```

```
# platform.properties: the transport when none is given
spring.profiles.default=cli
# application-confluent.properties: the cluster, the registry, SASL, the serializers; a missing key fails startup
spring.config.import=classpath:kafka.properties
spring.kafka.bootstrap-servers=${KAFKA_BOOTSTRAP_SERVERS}
```

```
make run TRANSPORT=confluent ENV=test    # SPRING_PROFILES_ACTIVE=confluent,test, the keys from .env*
```

● LIVE · make run-tiny · make bdd

Meet BDD

Cucumber · Gherkin · Testcontainers

Feature	Story	Reads	Puts
I can publish messages	the generator		products, offers, orders
At the tea party an order meets an offer	the tea party	offers, orders	transactions
Alice's purse knows what is left	the purse	transactions	

```
Scenario Outline: An order is settled with an offer for its thing
  Given a tea party in region <Region>
  And Mad Hatter offers a Top Hat for 12.5 coins
  And Alice orders a Top Hat
  When the tea party reads the streams of offers and orders
  Then Alice paid Mad Hatter 12.5 coins for the Top Hat
```

Out of trial credits :)

● LIVE · make kafka-universe-up · make run-tiny TRANSPORT=kafka · localhost:8082

The kafka universe

Apache Kafka · Confluent Schema Registry · Kafbat UI · compose.kafka.yaml

	Offset	Partition	Timestamp	Key Preview
−	0	4	29.10.2026, 02.48.45.542	EMEA/P-HEDG

Key | Value | Headers


```
{
  "offer_id": "OFF-ecae7aea-52d4-3bb5-b22a-9c7584544946",
  "product_id": "P-HEDG",
  "price": 86.29,
  "seller_id": "MARCH_HARE",
  "created_at": "2026-10-29T01:48:44.816766Z"
}
```

The very simple refactoring

kafka is the transport, confluent is kafka with credentials · five annotations, one group, the defaults for a broker next door

```
@Profile("kafka")   // was "confluent": KafkaPublisher, StoryListener, Reader, DeadLetters, Topics
```

```
# platform.properties: confluent plays kafka's adapters and settings underneath
spring.profiles.group.confluent=kafka
# kafka.properties: the broker and the registry of make kafka-universe-up, unless the environment names others
spring.kafka.bootstrap-servers=${KAFKA_BOOTSTRAP_SERVERS:localhost:9092}
spring.kafka.properties.schema.registry.url=${SCHEMA_REGISTRY_REST_ENDPOINT:http://localhost:8081}
# application-confluent.properties: what Confluent Cloud adds, SASL and the registry's basic auth
spring.kafka.security.protocol=SASL_SSL
```

```
@kafka
Feature: The cluster has my topics and schemas

  Scenario Outline: A topic exists with its schema            # plays on kafka and confluent
    Then the topic <Topic> exists
    And the Protobuf schema <Schema> is registered for the topic <Topic>

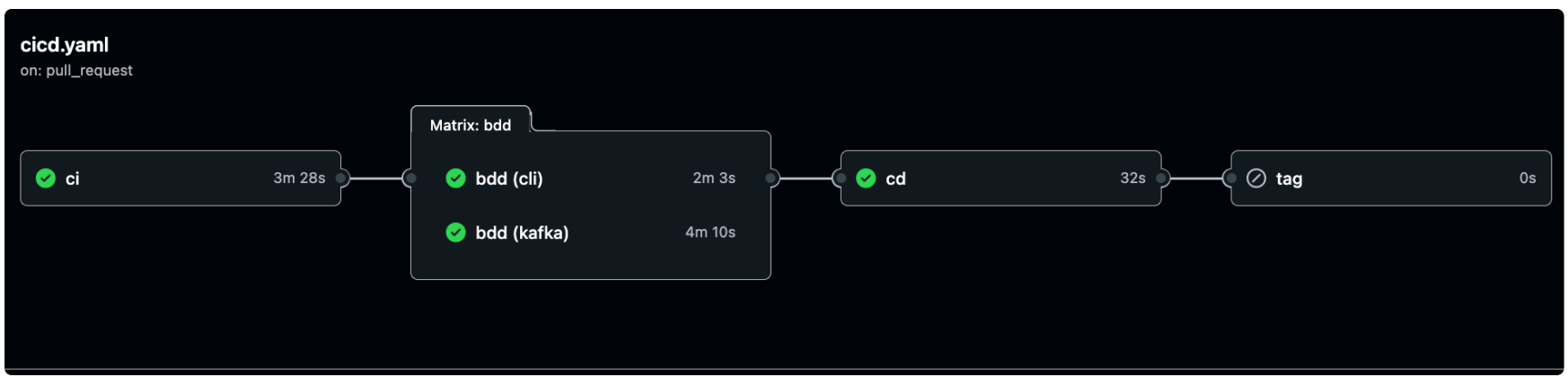
  @confluent
  Scenario Outline: A Flink table set up with IaC is on the cluster, with its schema    # confluent alone
```

Happy build

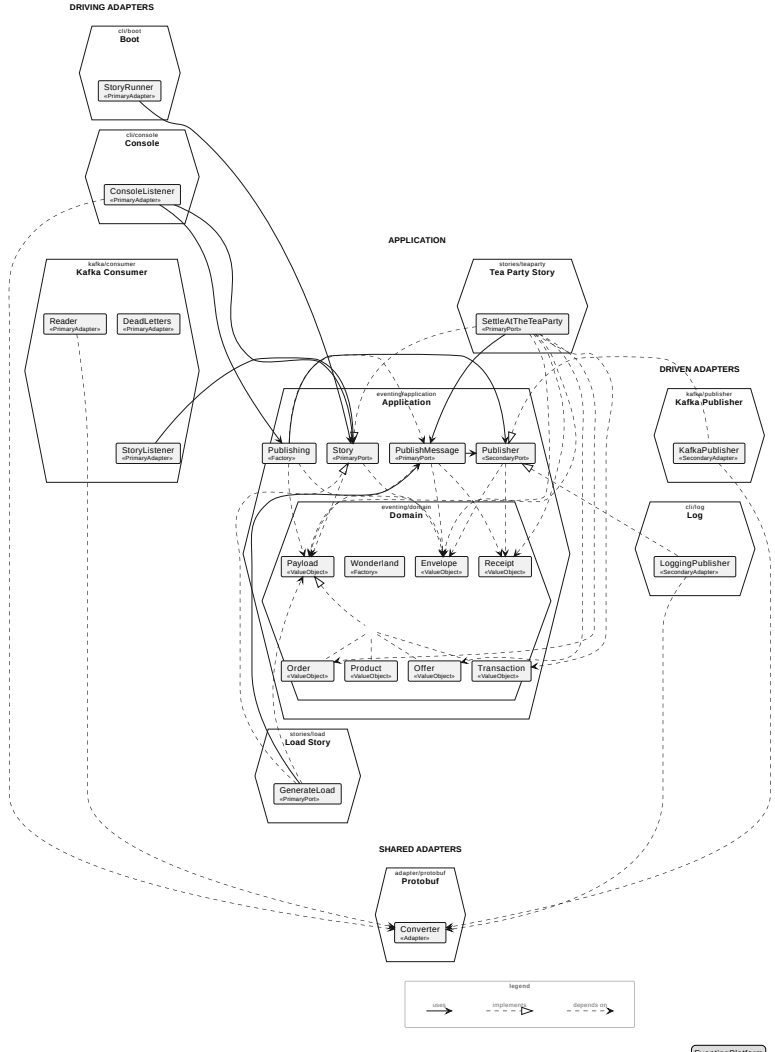
cli · kafka · cicd.yaml

```
make ci-transports            # ["cli","kafka"]; confluent sits beside it, commented out
make bdd                      # cli: the log and the console
make bdd TRANSPORT=kafka      # the universe up first, the containers on its network
make bdd-matrix               # both, locally what the bdd job plays
```

`cicd.yaml`, job `bdd`: `make bdd-published TRANSPORT=<t>` against the candidate image, one job per entry in `<t>-test`; `cd` publishes what both passed



● REC · cli, kafka, confluent



1 · Declare it in the code

The annotation is the spec

The platform's driving port, `Story`

```
@PrimaryPort
public interface Story {

    String name();

    default Set<Class<? extends Payload>> subscribesTo() {
        return Set.of();
    }
}
```

Its twin in the purse, a client's story, `KnowWhatIsLeft`

```
@PrimaryPort
public final class KnowWhatIsLeft implements Story {

    public static final String NAME = "purse";
```

2 · Verify it

Two rules, plain unit tests

Live: break the hexagon

3 · Clients get the checks for free

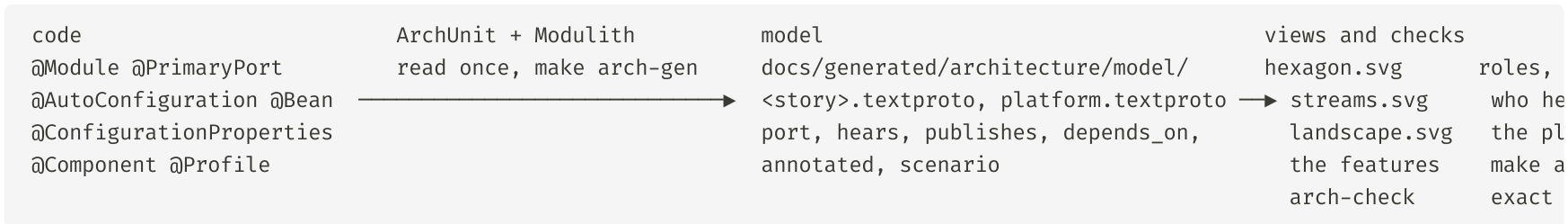
The purse extends the rules

The purse reuses the steps

4 · One model, three levels

Code in, model out

The build reads the classes once; everything else reads the model



hexagon.proto

● LIVE · make arch-gen · make arch-check · make arch-bdd

Express the architecture

purse.textproto · the_purse_has_hexagonal_architecture.feature

```
name: "Purse Story"
primary_port { name: "KnowWhatIsLeft"  implements: "Story" }
subscribes { payload: "Transaction" }
depends_on: ["Application", "Domain"]
wiring { configuration: "PurseStory"  settings { type: "PurseProperties"  prefix: "purse" }
         bean { name: "purse"  builds: "KnowWhatIsLeft"  from: ["PurseProperties"] } }
```

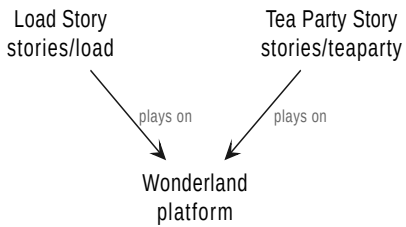
↑ the feature specifies arch-bdd checks ↓

```
@architecture-discovery
Feature: The purse has a hexagonal architecture

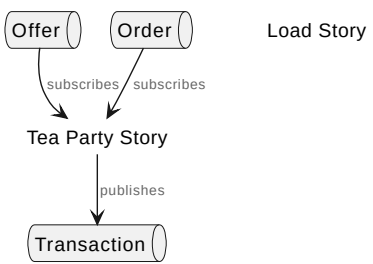
  Scenario: KnowWhatIsLeft is the purse's story
    Then its @PrimaryPort is KnowWhatIsLeft, which implements Story
    And KnowWhatIsLeft subscribesTo Transaction
    And KnowWhatIsLeft publishes nothing
```

Three levels

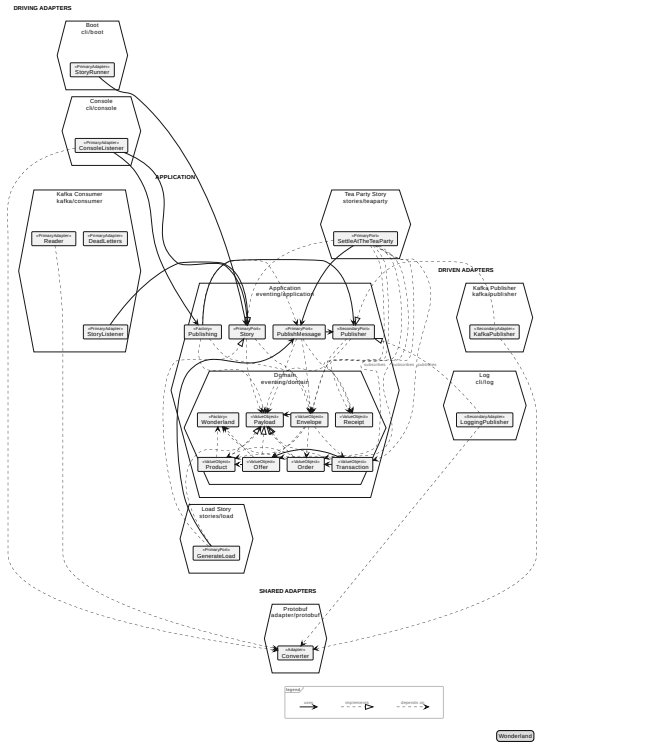
forest · streams · hexagon · `make arch-gen`



Level 1 · the forest · the platform and its stories · `make arch-forest`



Level 2 · the streams · a queue per topic, who publishes, who subscribes · `make arch-streams`



Level 3 · the hexagon · rings, modules, roles, arrows · `make arch-hexagon`

5 · The agent round trip

● LIVE · make claude STORY=examples/white-rabbit · make check

The agent round trip

`/wonderland-generate-story` · examples/white-rabbit

The agent gets	Where
the rules	README, Hexagon and Coding standards; `ArchUnitTest`, `ArchModulithTest`
the steps	the platform's test fixtures: `StreamSteps`, `HexagonSteps`, `WonderlandSteps`
the features	the_white_rabbit_has_no_time_to_haggle.feature · the_white_rabbits_hexagon.feature
the hexagon	white-rabbit.textproto
the platform	`make platform-install`; `examples/purse`, the worked example

```
primary_port { name: "NoTimeToHaggle"  implements: "Story" }
subscribes { payload: "Offer" }
publishes { payload: "Order"  through: "PublishMessage" }
```



Thanks! Questions?